

앙상블

Ensemble Model

앙상블 학습

■ 목적

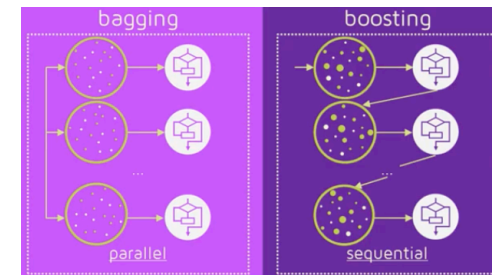
- 여러 분류기를 하나로 연결하여 개별 분류기 보다 더 좋은 일반화 성능을 달성하는 것

■ 방법

- 여러 분류 알고리즘 사용: 다수결 투표(Voting)
- 하나의 분류 알고리즘을 여러 번 이용: 배깅(Bagging), 부스팅(Boosting)

■ 종류

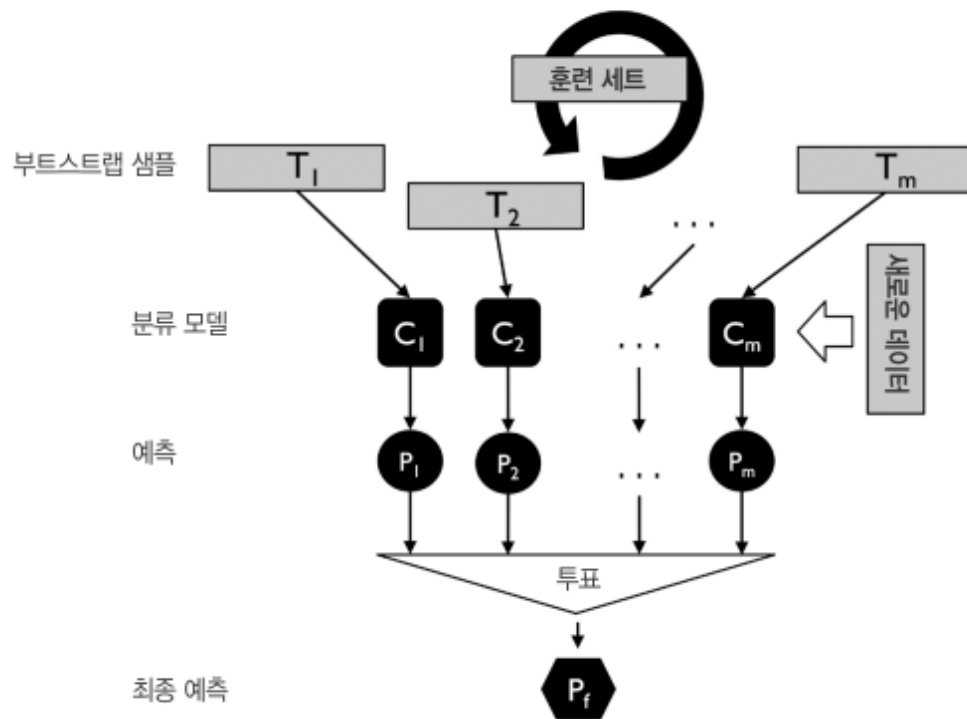
- 다수결 투표(Majority Voting)
 - 동일한 학습 데이터 사용
- 배깅(Bagging)
 - 알고리즘 수행 마다 서로 다른 학습 데이터 샘플링하여 사용
 - 병렬적 처리
- 부스팅(Boosting)
 - 샘플 뽑을 때 이전 모델에서 잘못 분류된 데이터를 재학습에 사용 또는 가중치 사용
 - 순차적 처리



배깅(Bagging)

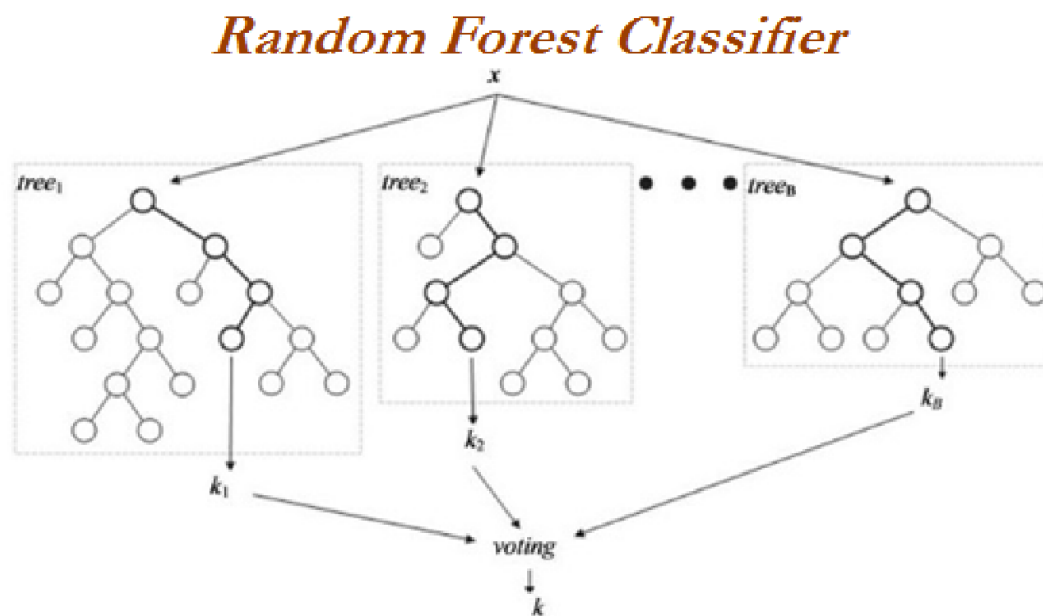
■ 배깅

- 알고리즘마다 별도의 학습 데이터를 추출(샘플링)하여 모델 구축에 사용
- 부트스트랩(Bootstrap) 사용
 - 학습데이터 샘플링 시 복원 추출(중복)을 허용



배깅(Bagging)

- 랜덤 포레스트 (Random Forest)
 - 배깅의 일종 : 학습 데이터 샘플링
 - 단일 분류 알고리즘(Decision Tree) 사용
 - Forest 구축: 무작위로 예측변수 선택하여 모델 구축
 - 결합 방식: 투표(분류), 평균화(예측)



배깅(Bagging)

- 랜덤 포레스트 (Random Forest) 분석 절차

1. 새로운 학습 데이터를 만든다.
 - 크기가 n 이고 d 개의 특성 변수를 가지는 학습 데이터
 - Bootstrap을 고려한 n 개의 새로운 학습 데이터 구성
2. 새로운 학습 데이터를 이용하여 의사결정나무를 완성한다.
 - d 개의 특성 변수 중 임의로 k 개의 특성 변수를 뽑아 의사결정나무 구성
 - k 는 일반적으로 $\sqrt{d}$ 를 선택함
3. 절차 1-2를 M 번 반복한다.
 - M 이 커질수록 성능이 좋아짐. 리소스를 고려하여 가능한 큰 M 을 선택
 - M 은 트리의 수

통계적 머신러닝의 특징

■ 적절한 알고리즘 선택 시 고려해야 할 것

1. 자료의 타입에 민감한가?
2. 결측 자료에 민감한가?
3. 자료의 이상 치에 민감한가?
4. 자료의 표준화가 필요한가?
5. 해석이 용이 한가?
6. 성능이 우수 한가?

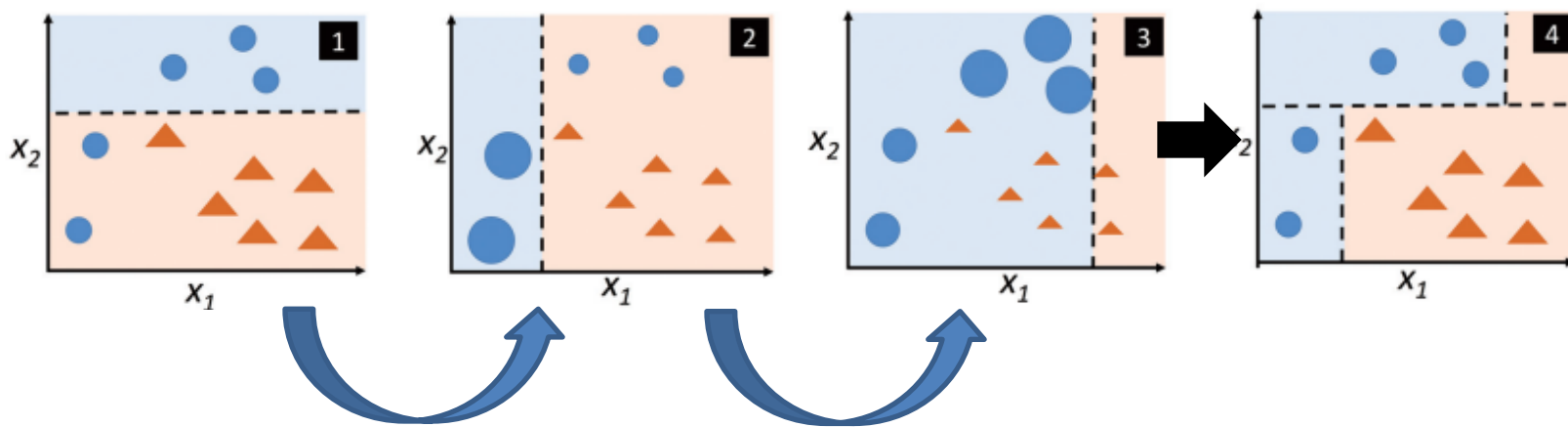
전처리 관련 요소

특성	로지스틱	KNN	LDA	SVM	의사결정나무	신경망
자료타입 민감성	상	상	상	상	하	상
결측 민감성	상	중	상	상	하	상
이상치 민감성	상	하	상	상	하	상
표준화 필요성	선택	선택	선택	선택	불필요	필요
해석의 용이함	용이	난해	난해	난해	용이	매우 난해
성능의 우수함	중	중	중	중	중	상

부스팅(Boosting)

■ 아다부스트 (AdaBoost)

- 순차적으로 weak learner를 적용하여 모델 구성하는 방법
 - weak learner의 조합을 통해 strong learner 만들기
- weak learner 란
 - 임의 분류(Random guessing)보다 약간 좋은 모델
- weak learner 추가 시
 - 전체 학습데이터를 사용
 - 잘못 분류된 데이터에 가중치 적용
 - 앞선 모형이 잘 풀지 못하는 Hard cases를 더 잘 풀어보기 위해



Ensemble: Gradient Boosting Machine(GBM)

- 앙상블: 기울기 부스팅(Gradient Boosting)
 - 회귀, 분류 문제를 위해 모두 사용 가능
 - Gradient Boosting = Gradient Descent + Boosting
 - Weak learner 추가를 통해 잔차(residual)를 예측하도록 하는 방법
 - 손실 함수
 - 회귀: Squared Loss, Absolute Loss, Huber Loss, Quantile Loss, etc.
 - 분류: Bernoulli Loss, Adaboost Loss, etc.
- 단점
 - 과적합에 빠지기 쉬움
 - 잔차를 모델링하다보니, 노이즈까지 모델이 반영하는 문제가 생김
 - 과적합 방지를 위한 해결책 (Regularization)
 - Subsampling (전체 데이터의 약 80% 사용함)
 - Shrinkage (잔차 모델의 영향력을 조금씩 줄이는 방법)
 - Early Stopping (검증 오차를 허용을 통한 과적합 방지법)

Ensemble: Gradient Boosting Machine(GBM)

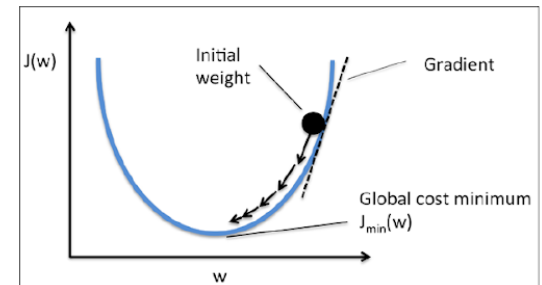
- 앙상블: 기울기 부스팅(Gradient Boosting)

- 회귀, 분류 문제를 위해 모두 사용 가능
- Gradient Boosting = Gradient Descent + Boosting
 - Loss function of least square

$$\min L = \frac{1}{2} \sum_{i=1}^n (y_i - f(x_i))^2$$

- Gradient of the loss function

$$\frac{\partial L}{\partial f(x_i)} = f(x_i) - y_i$$

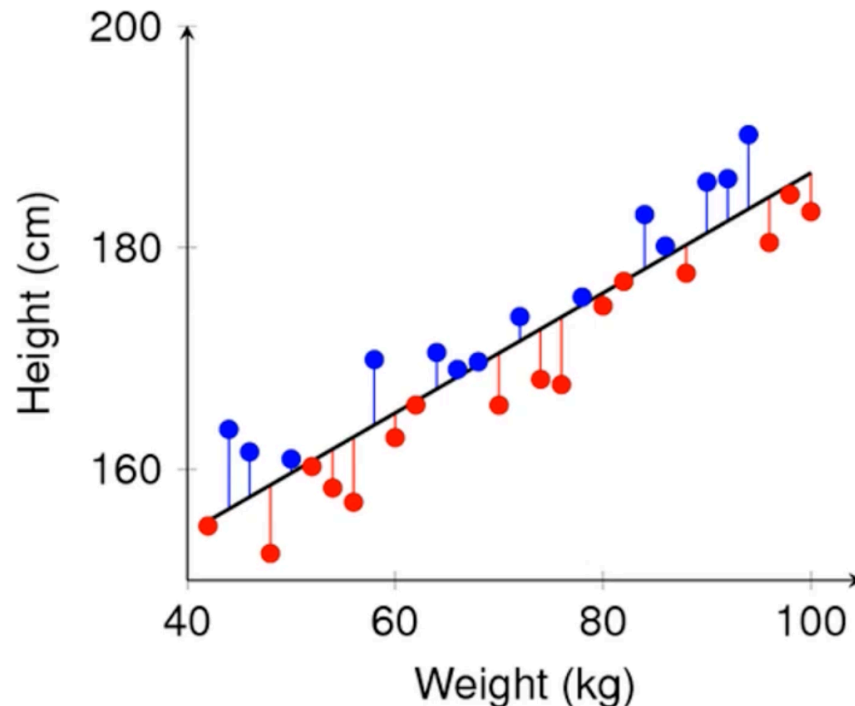


- Residual : negative gradient of the loss function ➔ 경사하강법!!

$$y_i - f(x_i) = -\frac{\partial L}{\partial f(x_i)}$$

Gradient Boosting Machine(GBM)

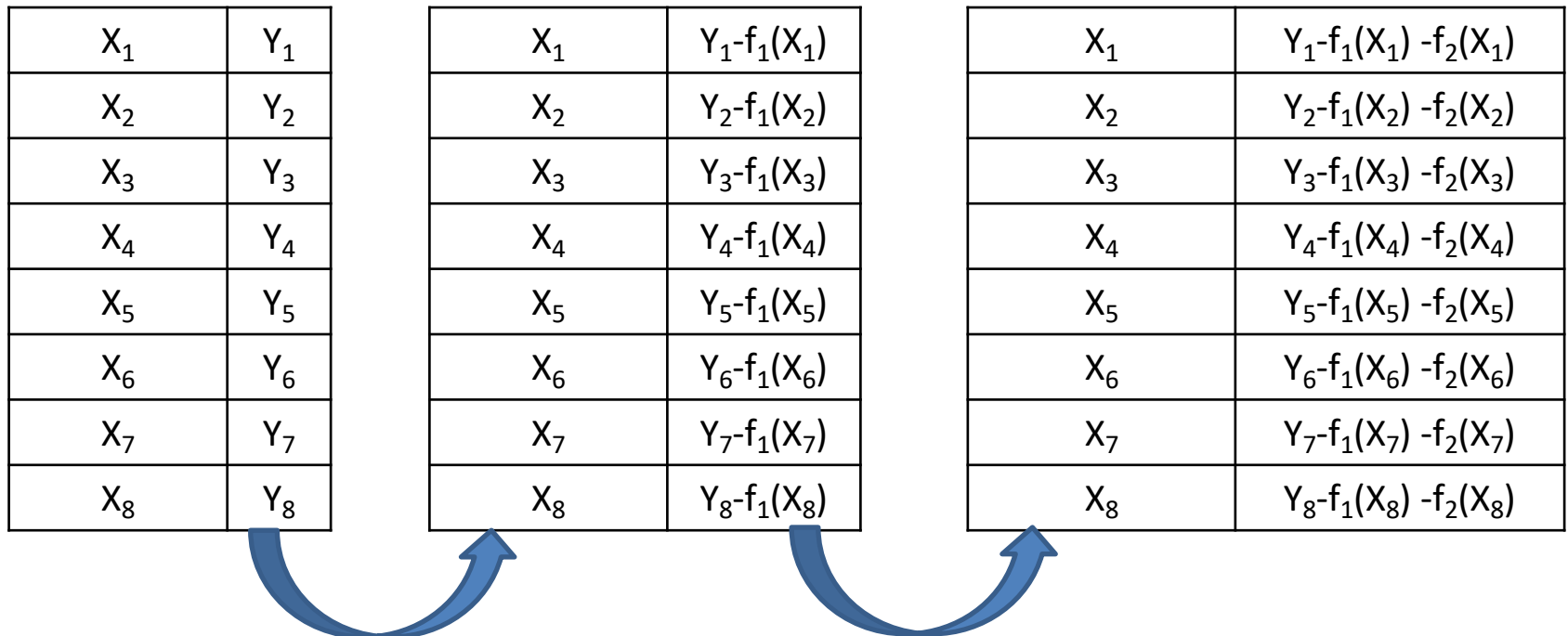
- 기울기 부스팅(Gradient Boosting)
 - Weak learner 추가를 통해 예측 잔차(residual)를 예측하도록 하는 방법
 - 순차적으로 모델을 적용함
 - 앞선 모델이 예측하지 못한 차이를 추가모델에서 보상하는 구조



Gradient Boosting Machine(GBM)

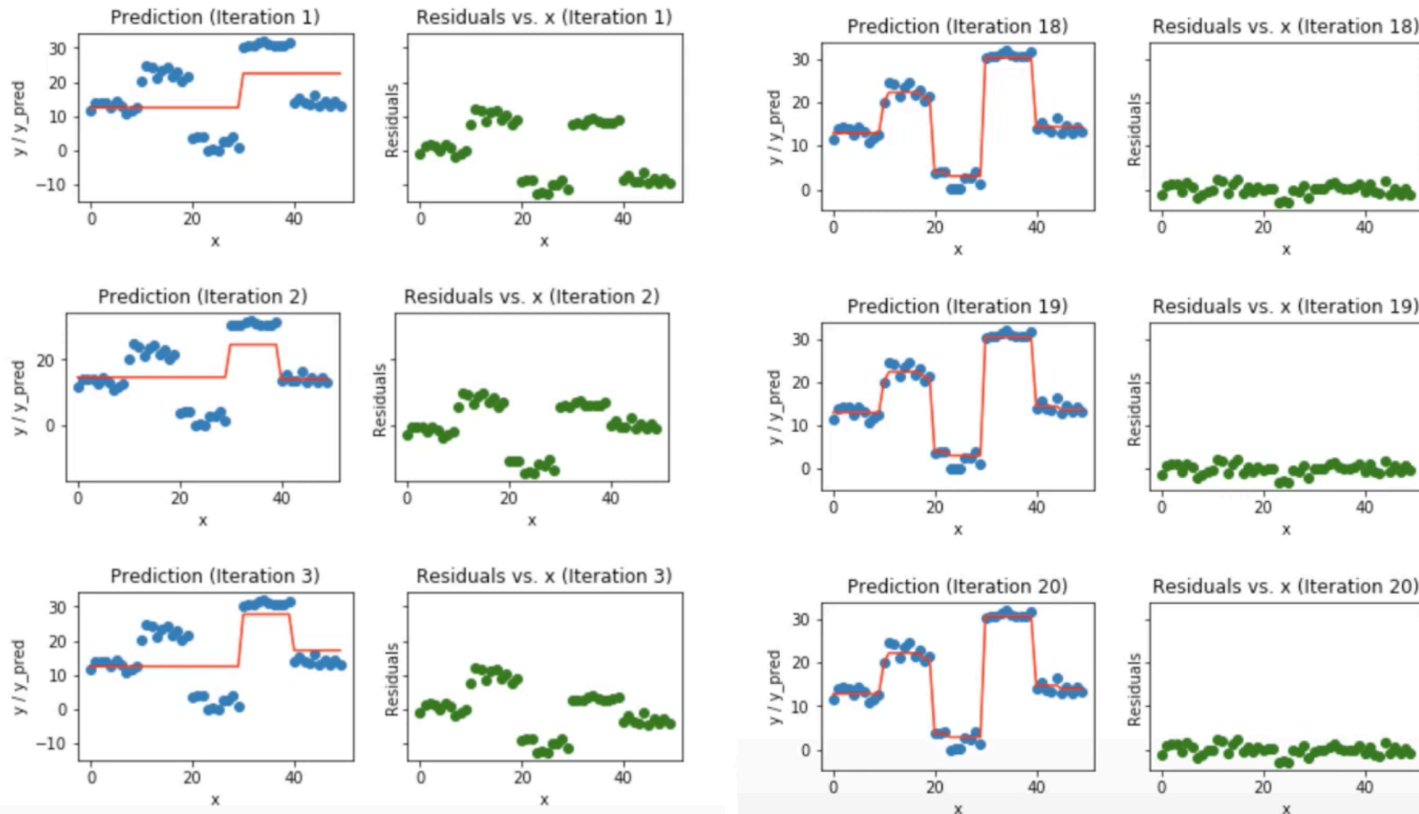
기울기 부스팅(Gradient Boosting)

- Weak learner 추가를 통해 예측 잔차(residual)를 예측하도록 하는 방법
 - 순차적으로 모델을 적용함
 - 앞선 모델이 예측하지 못한 차이를 추가모델에서 보상하는 구조



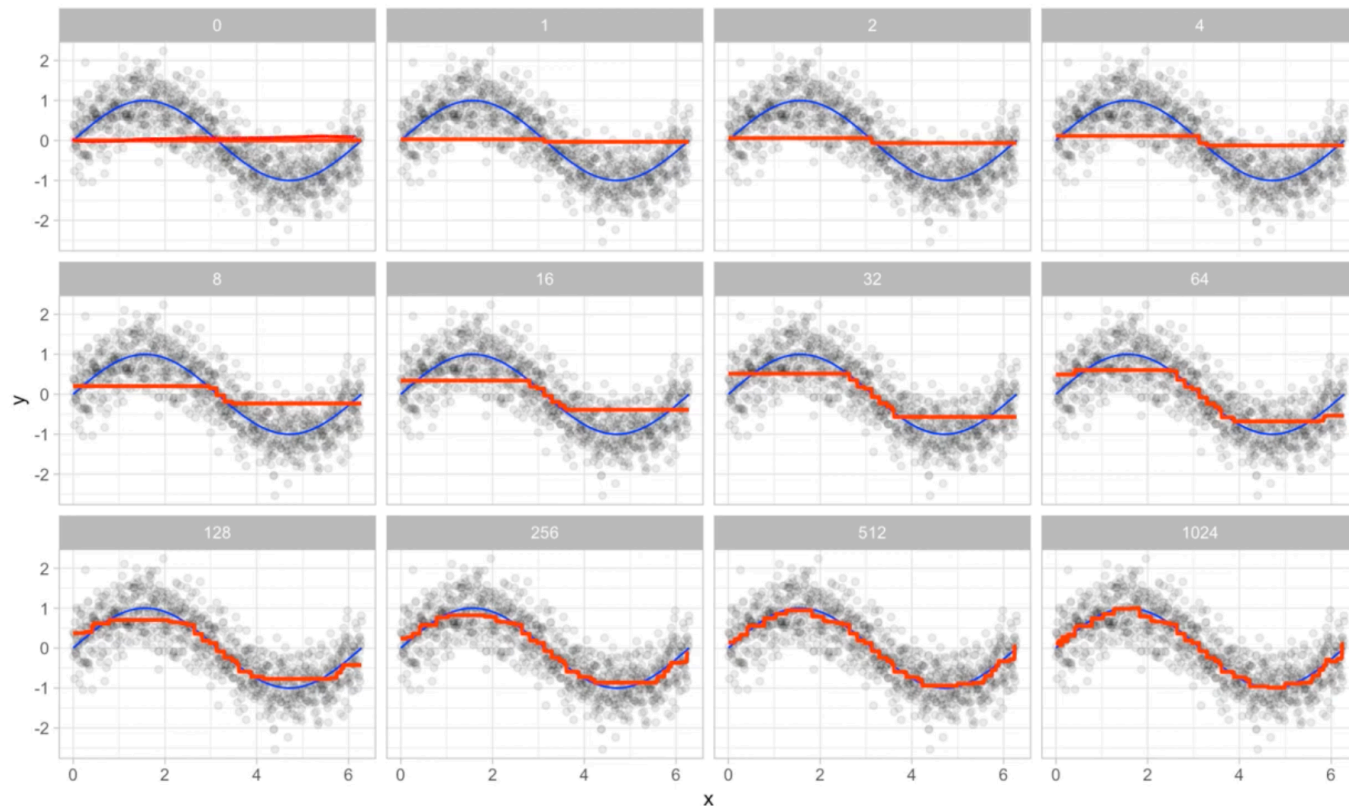
Gradient Boosting Machine(GBM)

- 기울기 부스팅(Gradient Boosting)
 - Weak learner 추가를 통해 예측 잔차(residual)를 예측하도록 하는 방법
- GBM Regression Example #1



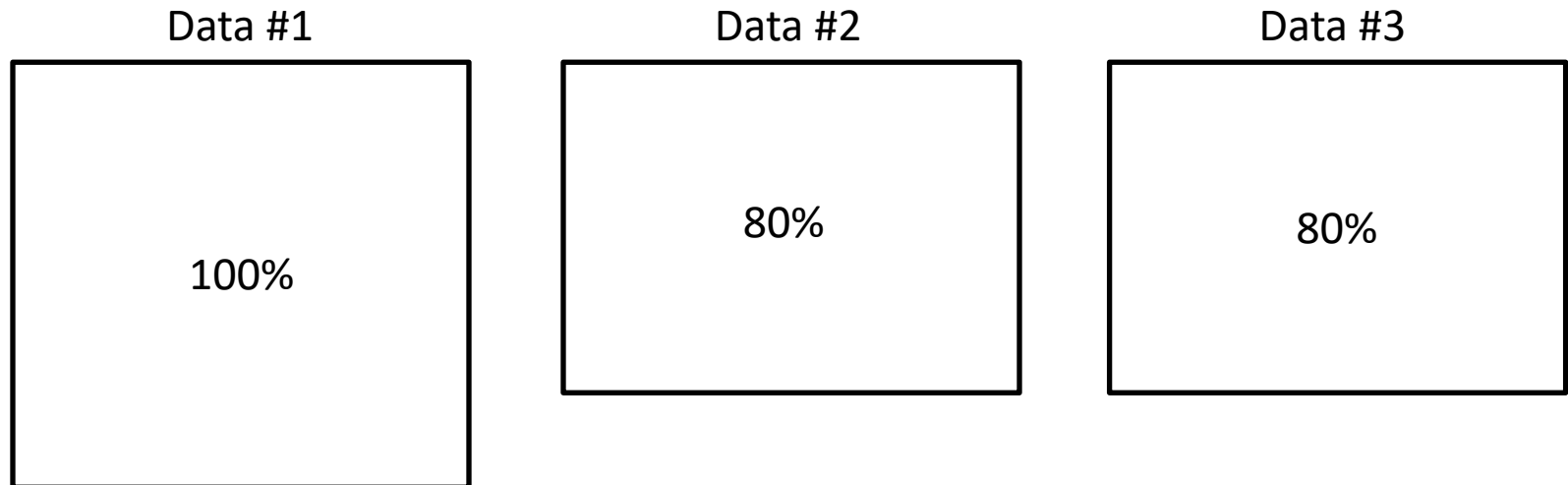
Gradient Boosting Machine(GBM)

- 기울기 부스팅(Gradient Boosting)
 - Weak learner 추가를 통해 예측 잔차(residual)를 예측하도록 하는 방법
- GBM Regression Example #2



Gradient Boosting Machine(GBM)

- 기울기 부스팅(Gradient Boosting)
 - 단점
 - 과적합에 빠지기 쉬움
 - 잔차를 모델링하다보니, 노이즈까지 모델이 반영하는 문제가 생김
 - 과적합 방지를 위한 해결책 (Regularization)
 - Subsampling (전체 데이터의 약 80% 를 반복적으로 사용함)
 - Replacement 여부 허용



Gradient Boosting Machine(GBM)

- 기울기 부스팅(Gradient Boosting)
 - 단점
 - 과적합에 빠지기 쉬움
 - 잔차를 모델링하다보니, 노이즈까지 모델이 반영하는 문제가 생김
 - 과적합 방지를 위한 해결책 (Regularization)
 - Subsampling (전체 데이터의 약 80% 를 반복적으로 사용함)
 - **Shrinkage (잔차 모델의 영향력을 조금씩 줄이는 방법)**
 - 추가 weak learner의 영향력이 점점 줄임

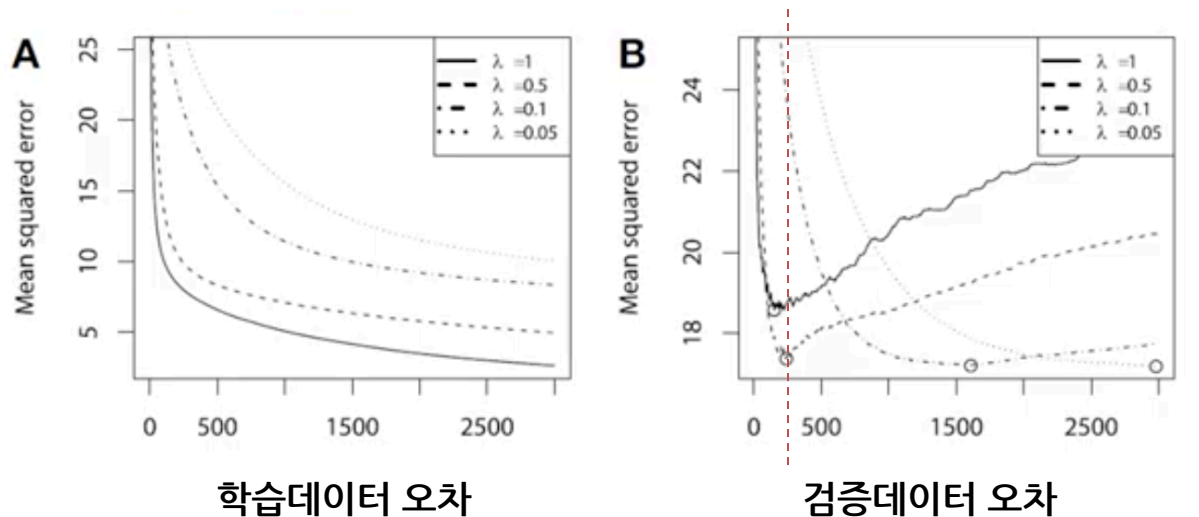
$$\hat{y} = f_1(x) + f_2(x) + \dots + f_{10}(x)$$



$$\hat{y} = f_1(x) + 0.9 * f_2(x) + 0.9^2 * f_3(x) + \dots + 0.9^9 f_{10}(x)$$

Gradient Boosting Machine(GBM)

- 기울기 부스팅(Gradient Boosting)
 - 단점
 - 과적합에 빠지기 쉬움
 - 잔차를 모델링하다보니, 노이즈까지 모델이 반영하는 문제가 생김
 - 과적합 방지를 위한 해결책 (Regularization)
 - Subsampling (전체 데이터의 약 80% 를 반복적으로 사용함)
 - Shrinkage (잔차 모델의 영향력을 조금씩 줄이는 방법)
 - **Early Stopping** (검증 오차를 허용을 통한 과적합 방지법)



LightGBM

- **LightGBM**

- 기존 GBM 동작 방식의 문제점

- 모든 데이터/모든 특성 변수에 대해서 알고리즘 수행 → 알고리즘 비효율적
 - 입력 데이터를 연산하기 효율적으로 변경

- 기존 GBM 동작 방식의 해결방법

- **Gradient-based One-Side Sampling(GOSS)**

- 모든 데이터를 사용하는 비효율성 해결책
 - Gradients가 큰 데이터 일 수록 영향을 크게 미치는 데이터 임
 - Small gradients는 랜덤 드랍, large gradient 만 포함

- **Exclusive Feature Bundling (EFB)**

- 모든 특성 변수를 사용하는 비효율성 해결책
 - 서로 거의 독립인 특성 변수(almost)를 묶어서 하나의 변수로 표현하는 방법
 - 데이터가 Very Sparse할 경우(예: one-hot encoding), EFB 를 적용하여도 성능 하락이 발생하지 않음

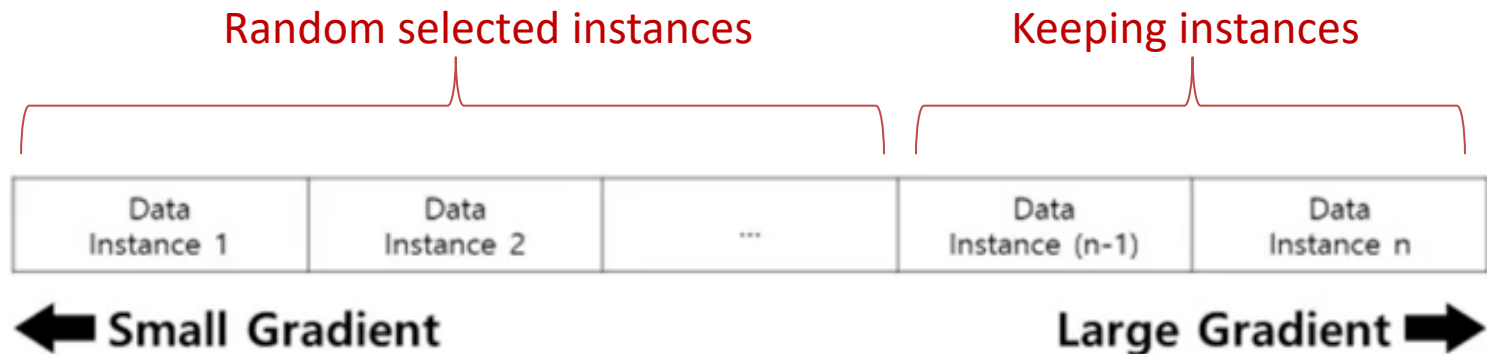
LightGBM

- LightGBM

- 기존 GBM 동작 방식의 해결방법

- **Gradient-based One-Side Sampling(GOSS)**

- 모든 데이터를 사용하는 비효율성 해결책
 - Gradients가 큰 데이터 일 수록 영향을 크게 미치는 데이터 임
 - Small gradients는 랜덤 드랍, large gradient 만 포함



LightGBM

- LightGBM

- 기존 GBM 동작 방식의 해결방법

- **Exclusive Feature Bundling (EFB)**

- 모든 특성 변수를 사용하는 비효율성 해결책
 - 데이터가 Very Sparse할 경우(예: one-hot encoding), EFB 를 적용하여도 성능 하락이 발생하지 않음
 - (1) 서로 거의 독립인 특성 변수를 묶어서, (2) 하나의 변수로 표현하는 방법
 - Greedy Bundling: 어떤 데이터를 하나로 묶을 것인가 (1)
 - Merge Exclusive Features: 하나로 묶어서 어떻게 표현할 것인가 (2)

LightGBM

- LightGBM
 - Exclusive Feature Bundling (EFB)
 - Greedy bundling example

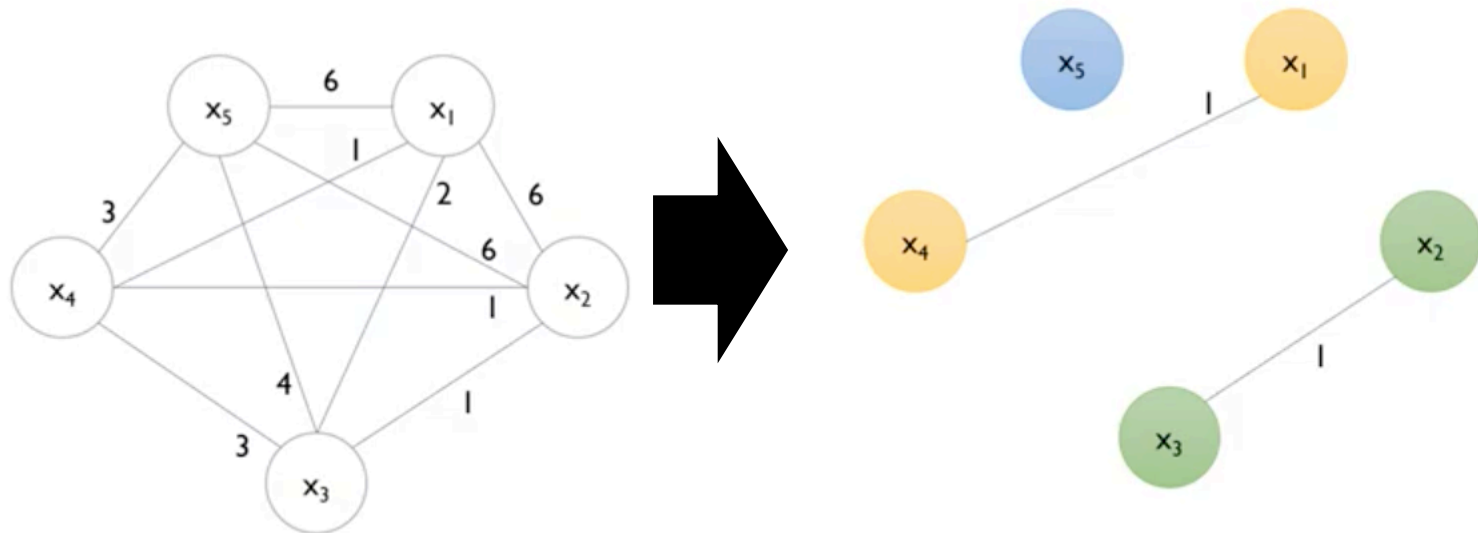
	x_1	x_2	x_3	x_4	x_5
I_1	1	1	0	0	1
I_2	0	0	1	1	1
I_3	1	2	0	0	2
I_4	0	0	2	3	1
I_5	2	1	0	0	3
I_6	3	3	0	0	1
I_7	0	0	3	0	2
I_8	1	2	3	4	3
I_9	1	0	1	0	0
I_{10}	2	3	0	0	2

	x_1	x_2	x_3	x_4	x_5
x_1	-	6	2	1	6
x_2	6	-	1	1	6
x_3	2	1	-	3	4
x_4	1	1	3	-	3
x_5	6	6	4	3	-

	x_5	x_1	x_2	x_3	x_4
d	19	15	14	10	8

LightGBM

- LightGBM
 - Exclusive Feature Bundling (EFB)
 - Greedy bundling example(cut-off =0.2)



Degree가 가장 높은 것 부터!

LightGBM

- LightGBM
 - Exclusive Feature Bundling (EFB)
 - Greedy bundling example(cut-off =0.2)

	x_1	x_2	x_3	x_4	x_5
l_1	1	1	0	0	1
l_2	0	0	1	1	1
l_3	1	2	0	0	2
l_4	0	0	2	3	1
l_5	2	1	0	0	3
l_6	3	3	0	0	1
l_7	0	0	3	0	2
l_8	1	2	3	4	3
l_9	1	0	1	0	0
l_{10}	2	3	0	0	2

	x_5	x_1	x_4	x_2	x_3
l_1	1	1	0	1	0
l_2	1	0	1	0	1
l_3	2	1	0	2	0
l_4	1	0	3	0	2
l_5	3	2	0	1	0
l_6	1	3	0	3	0
l_7	2	0	0	0	3
l_8	3	1	4	2	3
l_9	0	1	0	0	1
l_{10}	2	2	0	3	0

LightGBM

- LightGBM
 - Exclusive Feature Bundling (EFB)
 - Exclusive feature merging
 - Add offsets to the original values of the features

	x_5	x_1	x_4	x_2	x_3
l_1	1	1	0	1	0
l_2	1	0	1	0	1
l_3	2	1	0	2	0
l_4	1	0	3	0	2
l_5	3	2	0	1	0
l_6	1	3	0	3	0
l_7	2	0	0	0	3
l_8	3	1	4	2	3
l_9	0	1	0	0	1
l_{10}	2	2	0	3	0

	x_5	x_{14}	x_{23}
l_1	1	1	1
l_2	1	4	4
l_3	2	1	2
l_4	1	6	5
l_5	3	2	1
l_6	1	3	3
l_7	2	0	6
l_8	3	1	2
l_9	0	1	4
l_{10}	2	2	3